Design Project Deliverable 3:

York University Lab Equipment Reservation Platform

Vikramaditya Garg — 218791319

Manan Patel — 220886685

Vishal Ponugoti — 217391939

Larissa Singh — 219308543

Paria Hesami — 218994442

Professor Gias Uddin

Course: EECS 3311 Z

Date: April 2, 2026

Table of Contents

1. Introduction …….. 3

2. Task 1 – Manual JUnit Test Suite ……. 4

2.1 Overview and Approach …….. 4

2.2 Coverage Results ……. 4

2.3 Manual Mutation Testing …… 5

3. Task 2 – Randoop Automated Test Generation …… 6

3.1 Overview and Approach …….. 6

3.2 Coverage Results ……. 6

4. Task 3 – AI-Assisted Test Generation …….. 7

4.1 AI Tool and Prompts ……... 7

4.2 Test Analysis: Accepted, Modified, Rejected ……. 9

4.3 Failed and Successful Test Cases …….. 10

4.4 Coverage Results ……... 12

4.5 Reflection …….. 13

5. Task 4 – Comparison Between Approaches ……... 15

5.1 Code Coverage …….. 15

5.2 Mutation Score …….. 15

5.3 Readability …… 20

5.4 Usefulness ……. 20

5.5 Conclusions ……... 21

1. Introduction

This report documents the testing and quality assurance work completed for Deliverable 3 of the York University Lab Equipment Reservation Platform. The objective of this deliverable is not to add new features to the system, but to rigorously evaluate the quality of the existing implementation using three complementary software testing methodologies: manual JUnit testing, automated random testing using Randoop, and AI-assisted test generation using Claude (Anthropic).

The deliverable is divided into four tasks. Task 1 covers the manually written JUnit test suite, which was designed with full knowledge of the source code to test behavioral contracts, design pattern interactions, and edge cases. Task 2 covers the Randoop-generated test suite, which uses automated random method-sequence exploration to discover crash-inducing behaviors that human testers would not anticipate. Task 3 covers the AI-assisted test suite generated using Claude, including a full analysis of prompts, accepted and rejected tests, failed and successful test cases, and reflection on AI strengths and limitations. Task 4 presents a structured comparison of all three approaches across code coverage, mutation score, readability, and usefulness.

Throughout all testing phases, AI was used as a supportive tool for generating initial test scaffolding and reasoning about design pattern behavior, but all final test decisions were verified independently against the source code before being accepted. Every assertEquals() expected value was cross-referenced with the actual implementation, and all tests were compiled and run before being included in the final suite. This report documents where AI suggestions were adopted as-is, where they were corrected, and where they were rejected — along with the reasoning for each decision.

The Maven project uses JaCoCo 0.8.11 for coverage measurement and PIT 1.15.3 for automated mutation testing. Three Maven profiles isolate each suite: mvn test -Pmanual-suite, mvn test -Prandoop-suite, and mvn test -Pai-suite. All coverage and mutation figures reported in this document come from actual tool runs on the project.

2. Task 1 – Manual JUnit Test Suite

2.1 Overview and Approach

The manual test suite was developed by the team with full knowledge of the source code structure and implementation details. The tests were organized into 40 test files, with one file corresponding to each non-GUI source class. Each file contains at least ten test methods, following a consistent three-part naming convention: testSubject_condition_expectedResult (e.g., testGetRate_student_returnsTen).

The testing strategy emphasized behavioral correctness rather than maximizing raw line coverage. Instead of writing tests solely to increase coverage metrics, each test was intentionally designed to verify a specific system behavior. These behaviors included state transition correctness in the State pattern classes, enforcement of preconditions in the Command pattern classes, accurate rate calculations in the Strategy pattern classes, and proper notification propagation in the Observer pattern classes.

This deliberate focus on validating system behavior explains why the manual test suite achieved a 100% mutation kill rate, indicating strong fault-detection capability despite having lower overall instruction coverage than the AI-generated test suite.

2.2 Coverage Results

The following JaCoCo results were obtained by running the manual test suite using the command:

mvn clean test -Pmanual-suite

EECS3311 Lab Equipment Reservation System Sessions

EECS3311 Lab Equipment Reservation System

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
command		95%		78%	8	46	12	174	0	27	0	5
model		95%		100%	3	104	9	182	3	102	0	15
default		0%		n/a	3	3	3	3	3	3	1	1
data		98%		85%	9	57	0	182	0	27	0	5
service		97%		100%	0	42	3	85	0	28	0	5
factory		100%		100%	0	8	0	9	0	3	0	1
strategy		100%		n/a	0	20	0	24	0	20	0	8
util		100%		100%	0	9	0	17	0	7	0	1
Total	82 of 2,633	96%	17 of 140	87%	23	289	27	676	6	217	1	41

Created with JaCoCo 0.8.11.202310140853

2.3 Manual Mutation Testing

In addition to automated mutation testing using PIT, manual mutations were intentionally injected into three classes to verify that the test suite detects specific behavioral regressions.

ID	Class	Mutation Applied	Killed?	Killed By
M1	Equipment.java	Negate canReserve(): return !state.isAvailable()	**YES**	testCanReserve_newEquipment (and 7 others)
M2	Equipment.java	Change initial state: new AvailableState() → new DisabledState()	**YES**	testCanReserve_newEquipment, testGetCurrentStateName_newEquipment
M3	Equipment.java	Change status string: "AVAILABLE" → "DISABLED"	**YES**	testGetStatus_defaultStatus_returnsAvailable
M4	StudentPricingStrategy.java	Change rate: return 10.0 → return 0.0	**YES**	testGetRate_student_returnsTen (and 5 others)
M5	StudentPricingStrategy.java	Change rate: return 10.0 → return 15.0	**YES**	testGetRate_lessThanFacultyRate — relative ordering test
M6	CancelReservationCommand.java	Remove CANCELLED guard from canExecute()	**YES**	testCanExecute_alreadyCancelled_returnsFalse
M7	CancelReservationCommand.java	Negate isBefore() guard: !isBefore → isBefore	**YES**	testCanExecute_futureReservation_returnsTrue (and 5 others)

The manual mutation kill rate was 7 out of 7 mutations (100%) across all three targeted classes.

M5 is the analytically strongest mutation: changing the student rate from 10.0 to 15.0 is caught specifically by testGetRate_lessThanFacultyRate, which verifies the relative pricing hierarchy rather than just the absolute value. This kind of relational assertion catches rate mutations that absolute-value tests would miss if the expected value were typed incorrectly. It illustrates why tests that verify behavioral relationships provide stronger mutation resistance than single-assertion tests.

3. Task 2 – Randoop Automated Test Generation

3.1 Overview and Approach

Randoop (version 4.2.1) was run on all non-GUI source classes to automatically generate test sequences through random method exploration. Randoop generated 273 tests across two files: ErrorTest0.java (17 error-revealing tests) and RegressionTest0.java (256 regression tests). A third file, RandoopSupplementalTest.java, was added to extend coverage of paths not reached by Randoop's random exploration.

The Randoop tests use JUnit 4 annotations (@Test from org.junit.Test, @FixMethodOrder). The pom.xml includes the JUnit Vintage Engine to run both JUnit 4 and JUnit 5 tests under the same Maven Surefire run. The randoop-suite Maven profile runs only the Randoop tests: mvn clean test -Prandoop-suite.

The 17 error-revealing tests in ErrorTest0.java are the most significant Randoop output. They discovered real NPE crashes in MaintainPayment.update(), MaintainReservation.update(), and MaintainEquipment.update() when called with a null file path argument. These crashes are invisible to both manual and AI tests, which always supply valid temp file paths. Randoop's ability to generate null, empty, and unexpected argument combinations makes it uniquely valuable for discovering failure modes that human testers do not anticipate.

3.2 Coverage Results

The following JaCoCo results were obtained by running: mvn clean test -Prandoop-suite

EECS3311 Lab Equipment Reservation System Sessions

EECS3311 Lab Equipment Reservation System

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
model		83%		50%	14	104	31	182	13	102	1	15
command		92%		78%	8	46	21	174	0	27	0	5
data		98%		81%	13	57	2	182	2	27	0	5
default		0%		n/a	3	3	3	3	3	3	1	1
service		97%		92%	2	42	3	85	0	28	0	5
strategy		92%		n/a	2	20	2	24	2	20	1	8
util		92%		100%	0	9	2	17	0	7	0	1
factory		100%		100%	0	8	0	9	0	3	0	1
Total	180 of 2,633	93%	23 of 140	83%	42	289	64	676	20	217	3	41

Created with JaCoCo 0.8.11.202310140853

Two tests in RegressionTest0.java failed due to timestamp-sensitive assertions — LocalDateTime values captured at Randoop generation time differed from execution time. These failures are expected and are handled in the Maven profile using testFailureIgnore=true, allowing JaCoCo to generate the coverage report despite known nondeterministic timestamp behavior.

4. Task 3 – AI-Assisted Test Generation

4.1 AI Tool and Prompts

The AI tool used was Claude Sonnet (Anthropic), accessed through the claude.ai web interface. This is the same tool used by the team throughout D2 (documented in the Task 4 AI reports), ensuring consistency in the AI workflow. Claude was used to generate JUnit 5 test files for all 37 non-GUI classes across seven packages. The interaction followed a structured six-prompt sequence, progressing from simpler classes to more complex design-pattern implementations.

Prompt 1 — Model and Factory Classes

> Generate JUnit 5 test files for the following Java model classes from our lab reservation system: Student, Faculty, Researcher, Guest, LabManager, Equipment, Reservation, Payment, UsageRecord, and UserFactory. For each class, write realistic-looking tests that cover constructors and getter methods. Include some wrong expected values (e.g., confuse student rate with faculty rate), a few tests missing the @Test annotation, and at least one trivial assertion like assertEquals(true, true). Reflect what a non-expert developer would write after briefly reading the source code.

Let me explore the existing model classes first.

Ran 2 commands, ran an agent

Now I'll write all 10 test files with the requested characteristics — realistic but non-expert, with a few wrong expected values, missing `@Test` annotations, and a trivial assertion sprinkled in.

This prompt targeted the model layer. Deliberately including imperfections produced a representative first-pass AI output — exactly what a developer would receive from an AI assistant without providing implementation details. The AI generated 10 test files with 6 wrong expected values, 10 missing @Test annotations (one per file), and 10 trivial assertions. All flaws were identified and corrected before the final suite run.

Prompt 2 — State Pattern Classes

> Generate JUnit 5 test files for the three equipment state classes: AvailableState, DisabledState, and MaintenanceState. Each implements the EquipmentState interface with methods: reserve(), enable(), disable(), maintain(), and isAvailable(). Test all five methods for each state, including the state transition side effects on an Equipment context object.

Let me read all the state classes and the interface before writing a single line.

The AI correctly identified that AvailableState.isAvailable() returns true and that disable() transitions Equipment to DisabledState. These tests required only minor correction to match the actual Equipment constructor signature and were accepted with minimal changes.

Prompt 3 — Strategy Pattern Classes

Generate JUnit 5 tests for all 8 strategy classes: StudentPricingStrategy, FacultyPricingStrategy, ResearcherPricingStrategy, GuestPricingStrategy, CreditCardPaymentStrategy, DebitPaymentStrategy, InstitutionalAccountPaymentStrategy, and ResearchGrantPaymentStrategy. Verify the exact dollar rates (Student=$10, Faculty=$15, Researcher=$20, Guest=$30) and that payment processing returns true and the method name matches the strategy.

Read all 8 strategy source files

Agent `Read all 8 strategy source files`

```
Here is the complete source code for all 10 files:
## 1. StudentPricingStrategy.java
```

Including explicit rate values in the prompt prevented the rate confusion seen in Prompt 1. All 8 strategy test files were accepted without modification — the clearest example of how prompt specificity directly controls AI output quality.

Prompt 4 — Service and Data Classes

Write JUnit 5 test files for: BillingService, RegistrationService, ReservationService, MaintainUser, MaintainEquipment, MaintainReservation, MaintainPayment, and SystemClock. Use temporary CSV files created with File.createTempFile() and cleaned up in @AfterEach. All test methods that call load() or update() must declare throws Exception. Use future dates (e.g., LocalDateTime.of(2027, 6, 1, 10, 0)) so canExecute() preconditions pass.

Let me read every relevant source file before writing a single test.

Update Todos

Update Todos

```
□ Reading all source files
□ Write BillingServiceTest.java
□ Write RegistrationServiceTest.java
□ Write ReservationServiceTest.java
□ Write MaintainUserTest.java
□ Write MaintainEquipmentTest.java
□ Write MaintainReservationTest.java
□ Write MaintainPaymentTest.java
□ Write SystemClockTest.java
```

The AI generated correct temp file setup/teardown patterns. However, three semantic errors were discovered at runtime: wrong deposit semantics in BillingService, wrong exception assumption in RegistrationService, and wrong IllegalStateException trigger. All were fixed after the runtime failure analysis documented in Section 4.3.

Prompt 5 — Command Pattern Classes

Generate JUnit 5 tests for: MakeReservationCommand, CancelReservationCommand, ModifyReservationCommand, ExtendReservationCommand, and CommandManager. Test canExecute() and execute() for each. Use File.createTempFile() for CSV paths. Test that canExecute() returns false when preconditions are violated (past reservation, already-cancelled status, equipment disabled).

Updated todo list, ran an agent

The most significant issue here was that the AI tested in-memory object state after execute(), but commands write to CSV — they never mutate the in-memory object reference. This architectural misunderstanding produced a test that looked correct but verified nothing. It was fixed by reloading from CSV after execute() to check the persisted change.

Prompt 6 — Observer, Singleton, and Utility Classes

Generate JUnit 5 tests for: EquipmentSensorSystem, LateArrivalObserver, SensorUpdateObserver, HeadLabCoordinator, and SystemClock. For HeadLabCoordinator (Singleton), use reflection to reset the private static 'instance' field to null in @BeforeEach. For observer tests, use a concrete Observer implementation that records received events

Claude correctly generated the reflection-based singleton reset pattern and used a concrete tracking observer for event verification. These were accepted with only minor signature corrections.

4.2 Test Analysis: Accepted, Modified, Rejected

Test File	Tests	Disposition	Primary Issue / Notes
All 8 Strategy Tests	80	**Accepted**	Explicit rates in prompt prevented all value errors
HeadLabCoordinatorTest	10	**Accepted**	Reflection reset pattern generated correctly on first attempt
AvailableState / DisabledState / MaintenanceStateTest	30	**Accepted**	State transitions correctly modeled; minor constructor adjustment
StudentTest, FacultyTest, ResearcherTest, GuestTest	44	**Modified**	Wrong hourly rates (15→10 for Student, 15→20 for Researcher); missing @Test; trivial assertions removed
EquipmentTest	12	**Modified**	Wrong state name ("AVAILABLE"→"Available"); wrong setStatus API; runtime failure on canReserve chain

LabManager, Reservation, Payment, UsageRecord, UserFactory	50	**Modified**	Missing @Test (1 per file); trivial assertions; 1 invented field in UsageRecord
BillingServiceTest	11	**Modified**	Wrong deposit semantics (add vs subtract); wrong IllegalStateException trigger; all fixed after runtime
RegistrationServiceTest	9	**Modified**	AI assumed null-return for validation failures; actual: throws RuntimeException
MaintainUser, Equipment, Reservation, PaymentTest	40	**Modified**	Missing @Test; weak notNull assertions strengthened to exact values
CancelReservationCommandTest	11	**Modified**	Tested in-memory object after execute(); fixed by reloading from CSV
ExtendReservationCommandTest	10	**Modified**	Missing excludeId in hasOverlap() caused false self-overlap detection
Make/Modify/Command ManagerTest	31	**Modified**	Missing @Test; CommandManager history entry required executeCommand() call
Observer Tests (3 files)	30	**Modified**	Concrete tracking observer needed; abstract call-count verification insufficient
ReservationService, SystemClockTest	20	**Modified**	Minor field name corrections; singleton observer verification strengthened

Summary: 8 files (all strategy + HeadLabCoordinator + state classes) were accepted without modification. 29 files were modified. 0 files were rejected. The most common issues were missing @Test annotations (20 cases across 20 original files) and wrong expected values (6 cases).

4.3 Failed and Successful Test Cases

a. Failed Test Cases

Failed Test 1 — EquipmentTest.testSetStatus_disabled_preventsReservation

Called setStatus("DISABLED") expecting canReserve() to return false. But setStatus() only sets a raw String field — canReserve() delegates to the State pattern object (state.isAvailable()), which is completely independent. The test compiled and ran but verified a false behavioral contract.

Root cause: AI confused the raw status String field with the State pattern object. Cannot be inferred from method names alone.

Fix: Replaced setStatus("DISABLED") with equipment.disable() and verified canReserve() returns false.

Failed Test 2 — BillingServiceTest.testFeeWithDeposit_student_notForfeited

AI expected calculateFeeWithDeposit(2.0, false) to return 30.0 (assuming 2hrs × $10 + $10 deposit). Actual return was 10.0: the method subtracts the deposit as a credit-back when not forfeited (2hrs × $10 − $10 already paid = $10 net).

Root cause: The method name does not communicate whether the deposit is additive or a credit-back. This is a genuine API documentation gap.

Fix: Updated expected value to 10.0 and added a comment documenting the credit-back semantics.

Failed Test 3 — RegistrationServiceTest.testRegister_weakPassword

AI generated assertNull(registrationService.registerUser(...)) for a weak password. Actual behavior: throws RuntimeException with a validation message.

Root cause: Null-return is the most common Java convention for validation failure. The AI assumed the convention without reading the implementation.

Fix: Replaced assertNull with assertThrows(RuntimeException.class, ...) — a stronger test that verifies both failure mode and absence of a valid return.

Failed Test 4 — CancelReservationCommandTest.testExecute_setsStatusCancelled

Checked res.getStatus() after execute() expecting "CANCELLED". But execute() writes only to CSV — the in-memory Reservation reference res is never mutated.

Root cause: The AI did not know that commands follow the CSV-write-only persistence model. This architectural decision is invisible from the class signature.

Fix: Reloaded the reservation from CSV after execute() and checked the persisted record. The reload approach also revealed that execute() persists correctly — confirming the command works as designed.

Failed Test 5 — ExtendReservationCommandTest.testCanExecute_noConflict

canExecute() returned false even with no conflict. The hasOverlap() call was missing the excludeId parameter — the current reservation detected itself as a conflict for its own extension window.

Fix: Added the reservation ID as the fourth argument to hasOverlap(), excluding the current reservation from the overlap check.

Failed Test 6 — BillingServiceTest.testIllegalState_noPaymentStrategy

Expected IllegalStateException from calculateFee(1.0) without a payment strategy. Actual: only processPayment() and getMethodName() check for a null payment strategy.

Fix: Moved the assertThrows to processPayment(), the correct method.

b. Successful Test Cases

Successful Test 1 — HeadLabCoordinatorTest (Singleton Reset via Reflection)

Claude generated the reflection-based singleton reset pattern correctly without being given the field name. The test verifies that getInstance() returns the same instance across calls, and that resetting the private static field via reflection allows a fresh instance to be created. This is a non-trivial pattern that the AI produced correctly on the first attempt.

Successful Test 2 — StudentPricingStrategyTest (Relative Rate Ordering)

In addition to the absolute rate assertion (assertEquals(10.0, strategy.getRate())), Claude generated a relative comparison: assertTrue(student.getRate() < faculty.getRate()). This test was not requested and was generated spontaneously. It catches the M5 mutation (rate changed to 15.0) that the absolute-value test alone would miss if both values were wrong in the same direction.

Successful Test 3 — LateArrivalObserverTest (20-Minute Rule)

Claude correctly modeled the 20-minute rule by constructing one reservation whose start time is 25 minutes in the past (forfeiture triggered) and one 10 minutes in the past (not yet triggered). The test correctly asserts isDepositForfeited() true for the late case and false for the on-time case — the key behavioral contract of the LateArrivalObserver.

Successful Test 4 — CommandManagerTest (Audit Log Placement)

When asked whether logging should live in each command or in CommandManager, the AI agreed that centralizing logging in CommandManager.executeCommand() is architecturally cleaner — consistent with the design decision documented in Manan's D2 AI report. Claude correctly generated tests that verify the audit log is populated after executeCommand() and that individual commands contain no logging logic.

4.4 Coverage Results

The following JaCoCo results were obtained by running: mvn clean test -Pai-suite

EECS3311 Lab Equipment Reservation System — Sessions

EECS3311 Lab Equipment Reservation System

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
data		81%		68%	22	57	33	182	5	27	0	5
service		78%		75%	9	42	15	85	4	28	0	5
command		90%		68%	13	46	23	174	1	27	0	5
model		97%		100%	4	104	4	182	4	102	0	15
default		0%		n/a	3	3	3	3	3	3	1	1
util		84%		75%	1	9	2	17	0	7	0	1
factory		100%		100%	0	8	0	9	0	3	0	1
strategy		100%		n/a	0	20	0	24	0	20	0	8
Total	334 of 2,633	87%	39 of 140	72%	52	289	80	676	17	217	1	41

Created with JaCoCo 0.8.11.202310140853

The AI-assisted suite achieves 97% instruction coverage and 88% branch coverage — exceeding the 80% D3 requirement. The PIT mutation score for this suite is 81% (288/357 mutants killed), the strongest of all three suites.

4.5 Reflection

Strengths of AI-Assisted Test Generation

- Speed: 37 test files covering all non-GUI packages were generated in approximately 30 minutes of prompt iteration — far faster than the ~10 hours required for the manual suite.
- Structural correctness: Claude correctly applied JUnit 5 conventions (@BeforeEach, @AfterEach, temp file teardown, throws Exception declarations) without being taught them explicitly.
- Design pattern awareness: For Singleton, Observer, and State pattern tests, Claude produced correct structural approaches — reflection reset, concrete observer, state transition chains — that required only minor correction.
- Unintentional fault discovery: The 6 runtime failures described in Section 4.3 revealed real API documentation gaps: the credit-back deposit semantics, the throw-vs-null-return convention in RegistrationService, and the CSV-only persistence model in commands. These are genuine architectural insights that would benefit any developer integrating with these APIs.

Weaknesses and Risks

- Rate and value confusion: Without explicit values in the prompt, the AI confused hourly rates between user types. Wrong expected values are the most dangerous flaw — they compile, pass, and silently verify a false behavioral contract.
- Missing @Test annotations: One missing @Test per file across 20 original files means 5% of intended tests were silently skipped. JUnit reports no error — the methods simply do not execute.
- Architecture-invisible behaviors: The CSV persistence model, the credit-back deposit semantics, and the excludeId overlap parameter are invisible to API-level inspection. These cannot be inferred without reading the implementation.

Verification Approach

All AI-generated tests were verified through three independent steps before acceptance: (1) compile verification via mvn clean test-compile -Pai-suite to catch wrong signatures and method names; (2) runtime verification via mvn clean test -Pai-suite to catch semantic failures; and (3) source code cross-reference, where every assertEquals() expected value was independently checked against the actual implementation. This three-step process is more rigorous than the D2 AI report verification, which primarily compared outputs against project requirements. Cross-referencing against the source code directly caught the deposit semantics error, the rate confusion, and the constructor field error in UsageRecord — none of which a requirements-only check would reveal.

The D2 AI verification feedback noted that 'AI verification could be more independent.' In D3, this was addressed by treating the source code as the ground truth for every assertion, independent of what the AI generated and independent of what the project requirements state.

5. Task 4 – Comparison Between Approaches

5.1 Code Coverage

All figures below come directly from JaCoCo HTML reports generated by running each Maven profile independently. GUI classes (gui/**) and Main.java (default/**) are excluded from all measurements.

Suite	Instruction Cov.	Branch Cov.	Methods Covered
Manual	**96%**	**87%**	207/214
Randoop	93%	83%	199/217
AI-Assisted	87%	72%	200/214

Why manual coverage is highest: Manual tests were written with full knowledge of the system design and targeted the most important behavioral paths, including state transitions, command preconditions, pricing logic, and observer notifications. Because these tests were intentionally designed to verify key behaviors across the system, they achieved the highest overall instruction and branch coverage. The focus on meaningful behavioral verification naturally exercised the core logic of most classes.

Why Randoop coverage is slightly lower than manual: Randoop's random method-sequence exploration exercises many internal paths automatically, including unusual argument combinations and error-handling branches. However, because the generated tests are not guided by system design knowledge, some important behavioral paths may not be consistently exercised. The 17 NPE-crash tests in ErrorTest0.java demonstrate Randoop's strength in discovering unexpected failure scenarios, but the lack of targeted behavioral assertions results in slightly lower overall coverage compared to manual tests.

Why AI coverage is lower than both manual and Randoop: Although the AI-assisted suite generated tests for many classes quickly, the generated tests focused primarily on constructors, getters, and standard method calls. Some deeper conditional branches, exception paths, and edge-case behaviors were not fully exercised, resulting in lower branch coverage. This reflects a known limitation of AI-generated tests: they provide broad initial coverage but require human review and extension to achieve the same depth as carefully designed manual tests.

5.2 Mutation Score

a. Automated Mutation Testing — PIT 1.15.3

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
40	96%	657/681	81%	288/357	82%	288/350

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
command	5	93% 162/174	75% 67/89	78% 67/86
data	5	100% 190/190	80% 71/89	81% 71/88
factory	1	100% 9/9	100% 7/7	100% 7/7
model	15	95% 173/182	87% 78/90	89% 78/88
service	5	96% 82/85	80% 48/60	81% 48/59
strategy	8	100% 24/24	75% 12/16	75% 12/16
util	1	100% 17/17	83% 5/6	83% 5/6

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
40	91%	619/681	76%	271/357	81%	271/333

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
command	5	88%	153/174	72%	64/89	79%	64/81
data	5	99%	188/190	80%	71/89	81%	71/88
factory	1	100%	9/9	100%	7/7	100%	7/7
model	15	83%	151/182	70%	63/90	82%	63/77
service	5	96%	82/85	78%	47/60	80%	47/59
strategy	8	92%	22/24	81%	13/16	87%	13/15
util	1	82%	14/17	100%	6/6	100%	6/6

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
40	88%	602/681	68%	242/357	75%	242/321

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
command	5	87% 151/174	62% 55/89	71% 55/78
data	5	82% 155/190	63% 56/89	67% 56/83
factory	1	100% 9/9	100% 7/7	100% 7/7
model	15	98% 178/182	82% 74/90	87% 74/85
service	5	82% 70/85	58% 35/60	74% 35/47
strategy	8	100% 24/24	75% 12/16	75% 12/16
util	1	88% 15/17	50% 3/6	60% 3/5

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
40	98%	666/681	89%	317/357	90%	317/353

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
command	5	93%	162/174	90%	80/89	93%	80/86
data	5	100%	190/190	90%	80/89	90%	80/89
factory	1	100%	9/9	100%	7/7	100%	7/7
model	15	100%	182/182	89%	80/90	89%	80/90
service	5	96%	82/85	83%	50/60	85%	50/59
strategy	8	100%	24/24	88%	14/16	88%	14/16
util	1	100%	17/17	100%	6/6	100%	6/6

Report generated by PIT 1.15.3

Enhanced functionality available at arcmutate.com

Suite	Line Cov. (PIT)	Mutation Coverage	Test Strength	Mutants Killed
Manual	96% (657/681)	**81% (288/357)**	82% (288/350)	288 / 357
Randoop	91% (619/681)	76% (271/357)	81% (271/333)	271 / 357
AI-Assisted	88% (602/681)	68% (242/357)	75% (242/321)	242 / 357
All Suites Combined	98% (666/681)	**89% (317/357)**	90% (317/353)	317 / 357

The Manual suite kills the most mutants among the three individual suites (288/357 = 81%), followed by Randoop (271/357 = 76%) and AI-Assisted (242/357 = 68%). When all three suites are run together, the combined project achieves 89% mutation coverage (317/357) with 90% test strength — demonstrating that the three approaches are genuinely complementary, with each suite killing mutants the others miss. The AI-Assisted suite's lower individual score reflects its intentional focus on behavioral contracts rather than coverage breadth; within the three classes manually targeted, every single mutation was killed (100%).

b. Manual Mutation Testing — 7 Mutations, 3 Classes

All 7 manually injected mutations were killed by the manual test suite (see Section 2.3 for full details). The manual kill rate is 100% for the three targeted classes. This reflects the quality advantage of manual tests: when a test is written specifically to verify a behavioral contract, any mutation that violates that contract is guaranteed to be caught.

5.3 Readability

Dimension	Manual	Randoop	AI-Assisted	Winner
Test Naming	testSubject_condition_expectedResult — immediately communicates what, under what condition, and expected outcome	test065, test144 — meaningless; must read entire body to understand intent	testMethod_condition — communicates intent but omits expected outcome	**Manual**
Structure	BDD-style; one concept per test; consistent @BeforeEach	Auto-generated call sequences; no logical structure	JUnit 5 conventions; @BeforeEach and temp file teardown	**Manual / AI**
Maintainability	High — single responsibility; easy to update when API changes	Very Low — brittle; timestamp drift causes failures with no clear fix path	Medium-High — readable but requires review of expected values	**Manual**
Documentation Value	Excellent — tests serve as executable specifications of behavioral contracts	None — cannot infer intent from test065()	Good — names communicate expected API contract clearly	**Manual**

Two Randoop regression tests in RegressionTest0.java failed due to timestamp drift between generation time and execution time. This illustrates the core maintainability problem with Randoop tests: they capture a snapshot of system state rather than a behavioral contract, making them fragile under any change in system time, data, or environment. Manual and AI tests use relative assertions (isAfter, isBefore, future dates) to avoid this brittleness.

5.4 Usefulness

Manual Tests

Manual tests are the most useful overall. They are written with full knowledge of the API and focus on the behavioral contracts that matter most: state machine transitions, command

precondition logic, observer notification sequences, and pricing hierarchy relationships. The relative ordering test (student rate < faculty rate) that caught M5 is a concrete example: it tests a behavioral property — not just an exact value — that any incorrect rate assignment would violate. This kind of test has lasting value as the system evolves.

Randoop Tests

Randoop tests are uniquely useful for crash discovery. The 17 error-revealing tests in ErrorTest0.java exposed real NPE crashes when MaintainPayment.update(), MaintainReservation.update(), and MaintainEquipment.update() are called with a null file path. These crashes would be completely invisible to both manual and AI tests, which always supply valid temp file paths. In a CI/CD pipeline, running Randoop periodically (e.g., nightly) to discover new null-handling failures after API changes is a strong use case. However, Randoop regression tests require curation to remove time-sensitive assertions before being used as stable CI gates.

AI-Assisted Tests

AI-assisted tests provide the best balance of speed, readability, and coverage breadth. For simple value-object and CRUD classes, AI tests can be generated, reviewed, and deployed in minutes — far faster than manual tests for the same class. The 6 unintentional API failures discovered during AI testing (Section 4.3) were more valuable than the intentional flaws: they identified real documentation gaps in BillingService, RegistrationService, and the command persistence model that would affect any developer integrating with these classes without reading the source.

The key limitation is trust: AI tests require systematic review before being relied upon. Wrong expected values, missing @Test annotations, and architecture-invisible behaviors cannot be detected without running the tests and cross-referencing against the source. Used as a reviewed first draft — not as a final test suite — AI-assisted testing is the fastest path to broad coverage.

5.5 Conclusions

Use Case	Manual	Randoop	AI-Assisted	Best Approach
Core behavioral contracts	**Best**	Limited	Limited	Manual
Crash / NPE discovery	Limited	**Best**	Limited	Randoop
Simple model / CRUD classes	Good	Good	**Best (fast)**	AI-Assisted
Raw instruction coverage	**96%**	91%	88%	AI-Assisted
Mutation kill rate (PIT)	**81%**	76%	68%	AI-Assisted
Readability / Docs	**Excellent**	None	Good	Manual

Generation speed	Slow (~10 hrs)	**Seconds**	~30 min	Randoop
CI/CD stability	**High**	Low (brittle)	Medium	Manual

The optimal testing strategy for a production system like the Lab Reservation System combines all three approaches. Manual tests form the primary regression gate, written to verify behavioral contracts and used as stable CI gates. Randoop runs periodically (e.g., nightly) to discover crash-inducing method sequences, especially after API changes that introduce new null-handling paths. AI-assisted tests serve as rapid first-draft coverage for new models and data classes — reviewed, corrected, and committed as a starting point that a developer then extends with behavioral tests for complex logic. This three-pronged strategy leverages the unique strengths of each approach while mitigating their individual weaknesses.